



Systeme Intelligent de Classification des Incidents Aériens

Projet Cassiopee 2025

Équipe du projet :

Mounir Amanouz

Si-Belkacem Ketir

Othmane Belhaj

Encadrant :

Mounîm El Yacoubi

2 juin 2025

Table des matières

1	Introduction	3
2	Revue de l'état de l'art	4
2.1	Techniques de vectorisation [1] [3]	4
2.2	Modèles de classification pour données textuelles [1] [2] [3]	4
2.3	Techniques de gestion du déséquilibre des classes [4] [12]	4
2.4	Affinage de modèles préentraînés (Fine-tuning) [3] [10] [8]	5
2.5	Validation croisée (Cross-Validation) [2]	5
3	Contribution	6
3.1	Le jeu des données	6
3.2	Analyse exploratoire des données	6
3.2.1	Distribution et cooccurrence des classes	6
3.2.2	Longueur et qualité des rapports d'incident	7
3.2.3	Corrélations entre classes	8
3.2.4	Répartition des documents selon le nombre de problèmes abordés	8
3.3	Prétraitement des données	9
3.4	Tokenization et Embedding BERT	9
3.5	Entraînement et optimisation des modèles traditionnelles	10
3.5.1	Entraînement sur l'ensemble des données avec XGboost et LGBM	10
3.5.2	Undersampling et entraînement avec XGboost et LGBM	11
3.5.3	Entraînement à l'aide de la méthode <i>MAPIE</i> [13] [14]	12
3.6	Entraînement avec un modèle RNN	13
3.7	Affinement du modèle de langage BERT [9] [10]	14
3.8	Conclusion de notre contribution	14
4	Explicabilité avec SHAP [5] [6] [7] [11]	16
4.1	Motivation	16
4.2	Fondements théoriques	16
4.3	Visualisations SHAP	16
4.4	Exemple d'explication SHAP sur une prédiction (XGBoost)	16
4.5	Conclusion de l'explicabilité	17
5	Prototype de classification des rapports d'incident	19
5.1	Présentation du site web	19
5.2	Fonctionnalités principales	19
5.3	Résultats affichés après sélection du bouton " <i>Test and Predict</i> "	20
5.4	Résumé	22
6	Conclusion et Perspectives	23

1 Introduction

Dans le domaine de l’aviation, les opérateurs et les régulateurs sont confrontés à un afflux massif de rapports d’incidents rédigés par des humains. Cette abondance d’informations pose des défis majeurs en matière d’identification et de priorisation des incidents critiques, ainsi que dans l’assignation des équipes responsables de leur résolution.

Pour répondre à cette problématique, **nous développons un système de classification automatique permettant d’analyser et de catégoriser les descriptions d’incidents aéronautiques en fonction des problèmes rencontrés en vol**. Ce système est conçu pour comprendre le contenu textuel de chaque rapport et lui attribuer un ou plusieurs labels correspondant à environ 22 catégories spécifiques (par exemple : Engine Problem, Fire/Smoke Problem, etc.). **Cette solution vise à améliorer la gestion des incidents en facilitant leur traitement rapide et en générant des alertes pour limiter les immobilisations et prévenir les récides.**

Les données utilisées proviennent du jeu de données de la compétition SIAM 2007 Text Mining, publiée par la NASA (National Aeronautics and Space Administration) et accessible sur DATA.GOV. Cette compétition portait sur le développement d’algorithmes de classification de documents appliqués à des rapports de sécurité aérienne, décrivant un ou plusieurs incidents survenus en vol. Le jeu de données fourni, extrait du système public Aviation Safety Reporting System (ASRS) de la NASA, **comprend 21 519 rapports d’incidents pour l’entraînement et 7 077 pour le test**. L’objectif était d’étiqueter ces documents en fonction des types de problèmes mentionnés.

Dans ce projet, notre objectif est d’explorer et d’appliquer des techniques de data mining et de traitement automatique du langage naturel (NLP) pour analyser et exploiter des données textuelles. Nous cherchons à développer un système de classification fiable en tirant parti de modèles de machine learning, de deep learning, ou encore d’architectures avancées de grands modèles de langage (LLMs). Une attention particulière sera portée à l’intégration de méthodes d’explicabilité, afin de garantir une meilleure transparence dans les décisions prises par le système d’intelligence artificielle.

2 Revue de l'état de l'art

La première étape de notre projet a consisté en une exploration approfondie des méthodes de classification appliquées aux données textuelles. Dans ce cadre, nous avons été initiés aux principales techniques de traitement automatique du langage naturel. Nos recherches ont mis en évidence deux étapes fondamentales pour atteindre nos objectifs : d'une part, le prétraitement des données afin de les rendre exploitables par des modèles d'intelligence artificielle, et d'autre part, l'entraînement de ces modèles sur les données transformées.

2.1 Techniques de vectorisation [1] [3]

La vectorisation constitue une étape clé du prétraitement, permettant de transformer des textes en représentations numériques interprétables par les algorithmes. Plusieurs méthodes coexistent :

- **Méthodes classiques :**
 - *TF-IDF* (Term Frequency-Inverse Document Frequency) évalue l'importance d'un terme proportionnellement à sa fréquence dans un document, tout en atténuant l'effet des mots trop fréquents à l'échelle du corpus.
 - *Sac de mots* (Bag-of-Words) repose sur la représentation d'un texte par la fréquence d'apparition de chaque mot, sans tenir compte de l'ordre ni du contexte.
 - *Word2Vec* permet d'associer à chaque mot un vecteur dense, capable de capturer des relations sémantiques entre mots grâce à leur contexte d'apparition.
- **Méthodes contextuelles avancées :**
 - *BERT* (Bidirectional Encoder Representations from Transformers) est un modèle de type transformer qui considère simultanément les mots précédents et suivants pour capter le contexte global d'un mot.
 - *ALBERT* (A Lite BERT) constitue une version optimisée de BERT, qui réduit le nombre de paramètres via le partage de poids et la factorisation des représentations, tout en conservant de bonnes performances.

2.2 Modèles de classification pour données textuelles [1] [2] [3]

Dans le cadre de NLP plusieurs modèles d'intelligence artificielle :

- **Modèles classiques :** tels que *Naive Bayes*, *XGBoost*, *LightGBM*, *K-Nearest Neighbors (KNN)* ou encore *Naive Bayes Transfer*, appréciés pour leur rapidité et leur simplicité.
- **Modèles neuronaux :** comme les réseaux de neurones récurrents (*LSTM*) ou les modèles préentraînés comme *BERT* et *ALBERT*, qui offrent d'excellentes performances sur des corpus complexes.

Chaque type de modèle est adapté à des tâches en particulier : analyse de sentiments, classification d'actualités, reconnaissance d'entités nommées, ou encore extraction d'informations.

2.3 Techniques de gestion du déséquilibre des classes [4] [12]

Dans le domaine de la recherche, il est fréquent que la répartition des échantillons entre les différentes classes soit déséquilibrée, ce qui constitue un obstacle majeur pour les algorithmes de classification multi-label. En effet, les classes les moins représentées sont

souvent mal apprises par les modèles, ce qui entraîne une baisse de performance globale, en particulier en termes de rappel ou de couverture des labels rares. Pour faire face à ce défi, plusieurs stratégies ont été développées afin de rééquilibrer les données ou d'adapter les modèles d'apprentissage :

- **Approches fondées sur les données** : Ces méthodes visent à modifier la distribution des échantillons dans le jeu de données afin de réduire le déséquilibre entre les classes. Parmi elles :
 - *Le sous-échantillonnage* consiste à réduire le nombre d'exemples dans les classes majoritaires, ce qui permet de rapprocher leur fréquence de celle des classes minoritaires. Bien que cela puisse améliorer l'équilibre, cela risque de faire perdre une partie d'information précieuse.
 - *Le sur-échantillonnage* implique la duplication ou la génération synthétique d'échantillons des classes minoritaires (par exemple avec la méthode SMOTE), afin de leur donner plus de poids dans l'apprentissage.
 - *Les méthodes hybrides* combinent ces deux approches pour tirer parti de leurs avantages respectifs, tout en minimisant leurs inconvénients. Par exemple, un léger sous-échantillonnage couplé à un sur-échantillonnage contrôlé permet d'atteindre un compromis entre équilibre et diversité.

2.4 Affinage de modèles préentraînés (Fine-tuning) [3] [10] [8]

Au-delà des modèles traditionnels, il est possible d'adapter des modèles préentraînés à des tâches spécifiques grâce à un processus d'affinage, appelé *fine-tuning*. Cette méthode consiste à ajuster certains paramètres internes (taux d'apprentissage, poids, hyperparamètres, etc.) afin d'optimiser les performances du modèle sur une tâche cible. Contrairement à l'entraînement depuis zéro, le fine-tuning tire parti des connaissances déjà acquises par le modèle sur un corpus général, ce qui permet d'obtenir de bons résultats avec un volume de données réduit et un temps de calcul moindre. Ce procédé est particulièrement efficace dans des domaines spécialisés, comme le traitement du langage médical ou juridique, où les données annotées sont rares ou coûteuses à obtenir.

2.5 Validation croisée (Cross-Validation) [2]

La validation croisée est une méthode d'évaluation essentielle pour mesurer la performance d'un modèle de manière robuste et fiable. Elle consiste à diviser aléatoirement le jeu de données en plusieurs sous-ensembles appelés folds. Le modèle est alors entraîné sur une partie des données (généralement $k - 1$ folds) et testé sur le fold restant. Ce processus est répété k fois en changeant à chaque fois le fold utilisé pour la validation, ce qui permet d'obtenir une estimation moyenne des performances du modèle. Cette approche réduit le risque de surapprentissage (overfitting) et fournit une meilleure généralisation sur des données non vues. Elle est particulièrement utile lorsqu'on dispose d'un jeu de données limité, car elle permet d'exploiter l'ensemble des données à la fois pour l'entraînement et l'évaluation sans biais majeur. La validation croisée est souvent utilisée pour comparer des modèles ou ajuster des hyperparamètres de manière objective.

3 Contribution

3.1 Le jeu des données

Le jeu de données dont on dispose est constitué de quatre fichiers :

- **TrainingData.txt** : Contient les indices des documents ainsi que leurs textes décrivant les incidents. Ces rapports d'incidents seront utilisés pour l'entraînement (21519 échantillons).
- **TrainCategoryMatrix.txt** : Contient les étiquettes des documents d'entraînement sous forme de matrice, avec des valeurs 1 si l'étiquette correspond au document, et -1 sinon.
- **TestData.txt** : Contient les indices des documents ainsi que leurs textes décrivant l'incident. Ces rapports d'incidents seront utilisés pour le test (7077 échantillons).
- **TestCategoryMatrix.txt** : Contient les étiquettes des documents de test sous forme de matrice, avec des valeurs 1 si l'étiquette correspond au document, et -1 sinon.

Remarque : Le jeu de données dont nous disposons ne fournit aucune description explicite des étiquettes associées aux incidents. Autrement dit, nous ne connaissons pas la signification exacte des classes, qui sont simplement numérotées de 0 à 21. Nous devons donc considérer ces classes comme des identifiants anonymes, sans interprétation sémantique directe. L'apprentissage se fait ainsi dans un cadre de classification purement numérique.

3.2 Analyse exploratoire des données

3.2.1 Distribution et cooccurrence des classes

L'analyse préliminaire des rapports issus de notre base de données (**TrainingData.txt**) révèle une forte disparité dans la fréquence d'apparition des classes. La classe **1** domine largement avec plus de 12 000 occurrences. Elle est suivie par les classes **5** et **18**, qui apparaissent chacune environ 6 000 fois. D'autres classes telles que les classes **4**, **6**, **7**, **11** et **12** présentent une fréquence intermédiaire (entre 2 000 et 4 000 occurrences), tandis que le reste des classes est beaucoup moins représenté (moins de 1 000 occurrences), comme le montre la figure 1.

Par ailleurs, l'étude des cooccurrences (figure 4) met en évidence que la plupart des classes sont présentes dans des rapports multi-classes.

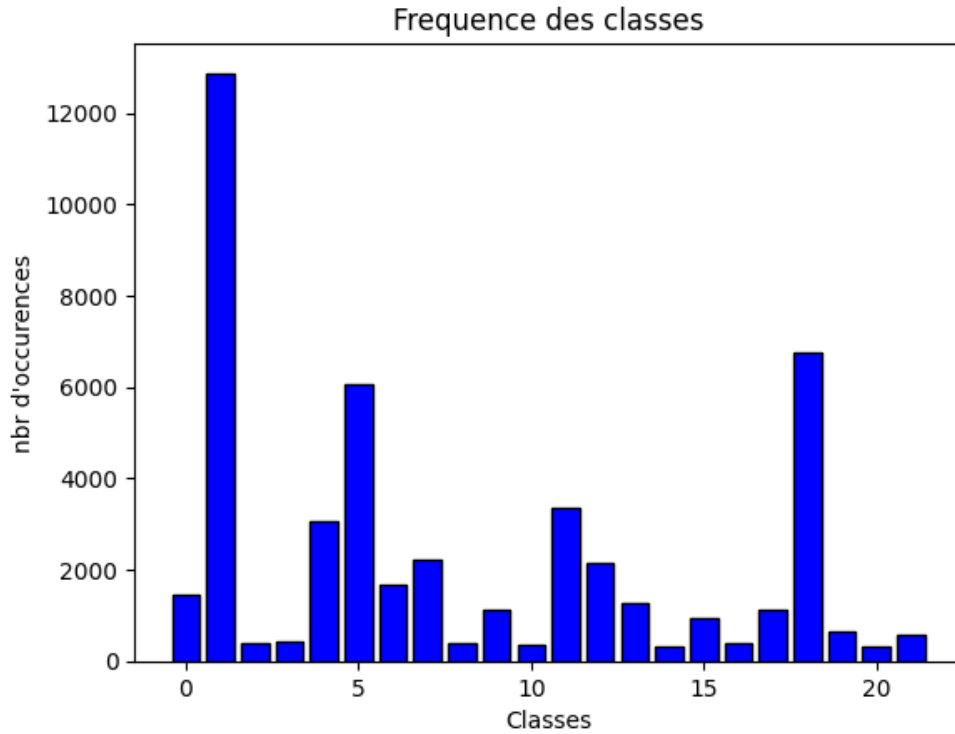


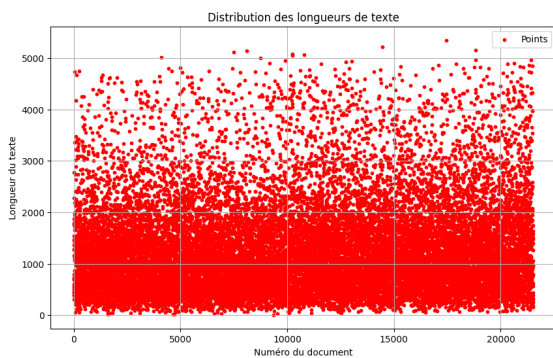
FIGURE 1 – Répartition des classes (fréquences individuelles)

3.2.2 Longueur et qualité des rapports d'incident

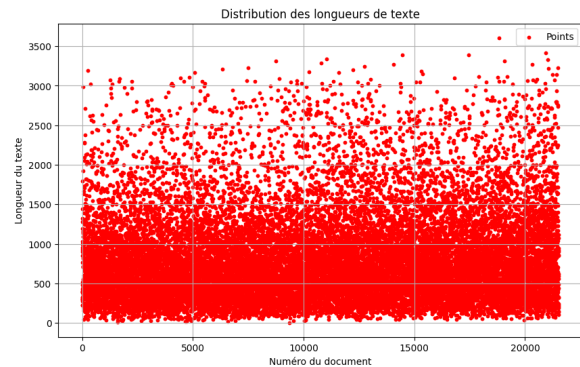
La longueur des rapports présente une forte hétérogénéité, avec une moyenne initiale de 1 288 mots, un minimum de 5 mots et un maximum atteignant 5 349 mots (figure 2a). Après l'application des étapes de prétraitement du texte — détaillées ultérieurement en section : *Prétraitement des données* — la longueur moyenne est ramenée à 822 mots, avec une valeur minimale de 4 mots et une maximale de 3 607 mots (figure 2b). Cette variabilité importante dans les longueurs complique le traitement automatique des données textuelles.

En plus de leur longueur, les rapports présentent plusieurs caractéristiques linguistiques particulières :

- des mots souvent collés sans espace,
- une grammaire imparfaite (notamment des erreurs de conjugaison),
- et une transformation systématique des chiffres en caractères soulignés ().



(a) Distribution des longueurs des rapports avant prétraitement



(b) Distribution des longueurs des rapports après prétraitement

FIGURE 2 – Comparaison des longueurs des rapports avant et après prétraitement

3.2.3 Corrélations entre classes

Enfin, nous avons examiné les corrélations entre les différentes classes. Comme le montre la figure 3, ces corrélations sont généralement faibles, ce qui suggère que chaque classe véhicule des informations relativement spécifiques. Cela justifie l'intérêt de traiter les classes de manière individualisée dans le modèle d'apprentissage supervisé.

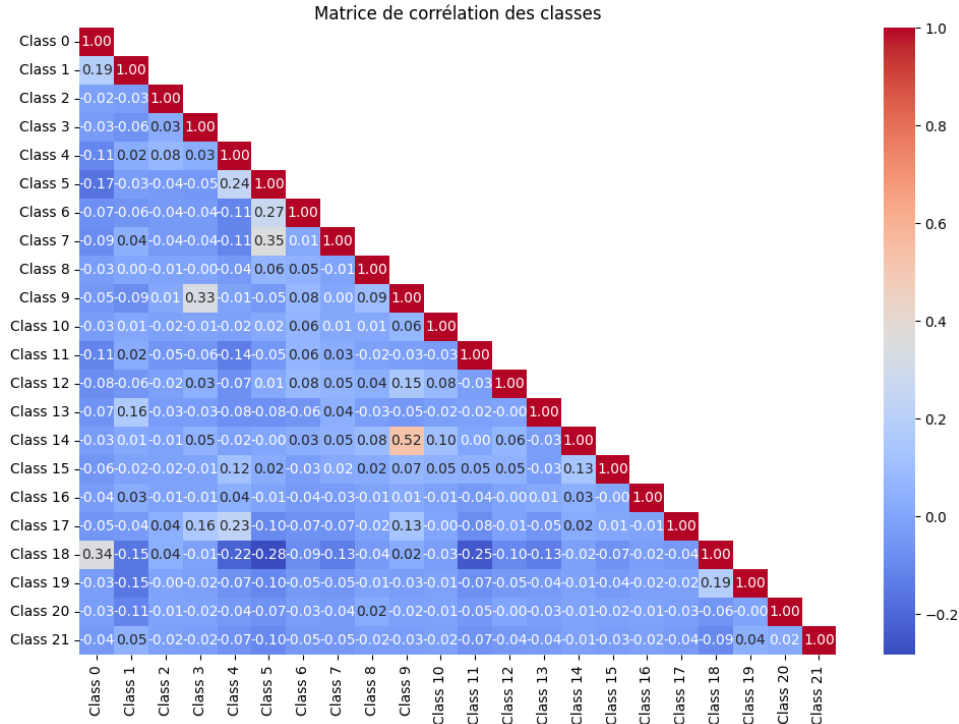


FIGURE 3 – Corrélations observées entre les classes

3.2.4 Répartition des documents selon le nombre de problèmes abordés

Nous avons cherché à savoir combien de documents traitent d'un seul problème, de deux, ou plus généralement, de N problèmes.

L'analyse montre que la majorité des documents couvrent entre 1 et 3 problèmes. Plus précisément et selon la figure 4 :

- 4862 documents traitent d'un seul problème,
- 5613 documents abordent deux problèmes (le nombre le plus élevé),
- 3813 documents en couvrent trois.

Au-delà de trois problèmes, le nombre de documents diminue fortement. Par exemple, comme le montre la figure 4 seulement 313 documents décrivent cinq problèmes, et les cas comportant six problèmes ou plus sont très rares.

Cette distribution montre que les incidents documentés sont le plus souvent causés par plusieurs facteurs, généralement deux ou trois. Toutefois, un incident peut aussi résulter d'un seul facteur. Les cas très complexes, impliquant plus de quatre causes distinctes, restent relativement rares.

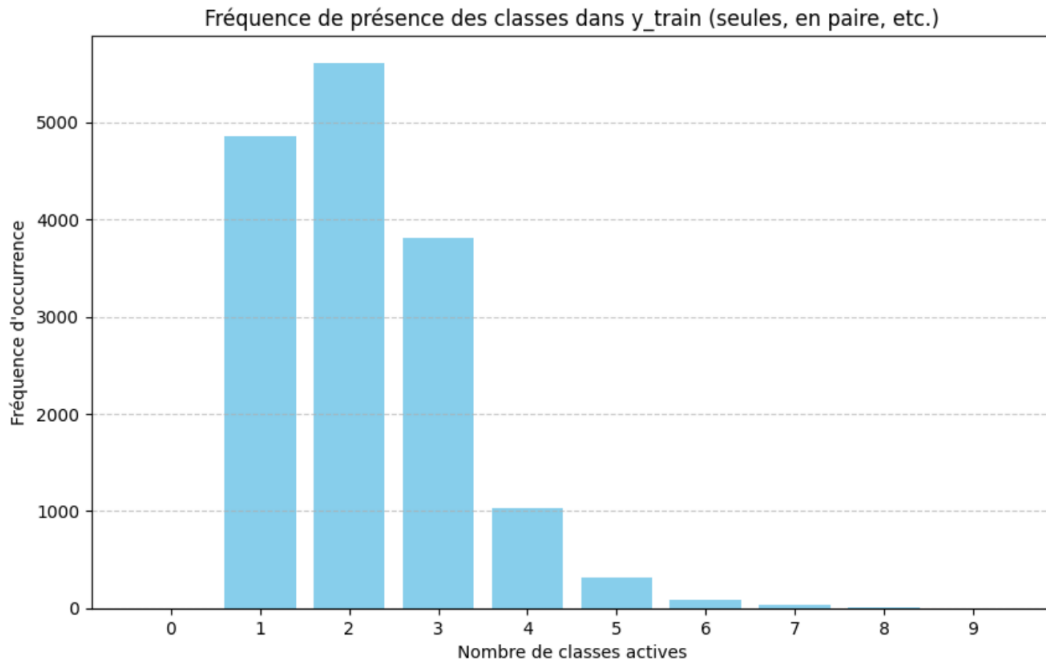


FIGURE 4 – Répartition des rapports selon N problèmes

3.3 Prétraitement des données

Le texte a été nettoyé selon plusieurs étapes :

- Suppression des valeurs manquantes,
- Suppression des caractères spéciaux et des chiffres,
- Suppression des mots vides (*stopwords*),
- Lemmatisation des mots pour les ramener à leur forme de base.

Après ce prétraitement, la longueur des textes a été modifiée, avec une moyenne désormais de 821 mots, un minimum toujours de 4 et un maximum inchangé à 3 607 selon la figure 2b.

La transformation appliquée au texte est démontrée dans notre application et elle est illustrée ultérieurement dans ce rapport, elle est représentée à la figure 9.

3.4 Tokenization et Embedding BERT

Dans un premier temps, nous voulons évaluer les performances de modèles classiques de machine learning. Pour ce faire, il a fallu vectoriser les textes, malgré les nombreuses imperfections de rédaction : fautes de frappe, mots collés, erreurs grammaticales, etc.

Afin de générer des représentations vectorielles robustes, nous avons choisi d'utiliser BERT, dont la tokenisation par sous-mots (WordPiece) permet de traiter efficacement les textes bruités. Nous nous sommes appuyés sur l'implémentation disponible dans le dépôt GitHub

Le prétraitement des textes s'effectue via une fonction dédiée qui :

- ajoute les tokens spéciaux [CLS] (début) et [SEP] (fin),
- tokenise le texte avec le tokenizer BERT,
- convertit les tokens en identifiants numériques,
- crée un vecteur de segment IDs,
- retourne les tokens, leurs identifiants et les segment IDs.

Une fois les données préparées, elles sont passées au modèle BERT pour générer les embeddings. La fonction dédiée :

- découpe les textes dépassant 512 tokens en segments,
- exécute BERT en mode inférence pour chaque segment,
- extrait les états cachés du dernier bloc,
- effectue une moyenne des embeddings des tokens pour chaque segment,
- puis une moyenne globale si plusieurs segments sont présents.

Le vecteur final, de dimension 768, constitue une représentation dense et sémantiquement riche du texte, prêt à être utilisé dans les tâches de classification ou de réduction de dimension.

3.5 Entraînement et optimisation des modèles traditionnelles

Nous avons ensuite entraîné plusieurs modèles de machine learning adaptés à la classification multi-label de texte, notamment **XGBoost** et **LightGBM (LGBM)**, cités dans la littérature comme efficaces pour ce type de tâche. **L'entraînement a été réalisé sur l'ensemble des données** avec une validation croisée et une recherche des meilleurs hyperparamètres.

Avant la phase d'entraînement, nous avons procédé à une séparation des données issues du fichier `TrainingText.txt` en deux ensembles distincts : un ensemble d'entraînement et un ensemble de test. Cette séparation a été effectuée à l'aide d'un *stratified split*, avec un ratio de 20% pour l'ensemble de test. Cette méthode garantit que la distribution des étiquettes multi-label est proportionnellement conservée dans les deux ensembles, assurant ainsi une représentativité équilibrée des classes dans l'évaluation du modèle.

Pour améliorer la robustesse de nos résultats et tirer parti de l'ensemble des données disponibles, nous avons utilisé la **validation croisée à 5 plis**. Cette méthode consiste à diviser le jeu de données en cinq sous-ensembles, puis à entraîner le modèle cinq fois en utilisant à chaque fois un pli différent pour la validation et les autres pour l'entraînement.

Cette approche est particulièrement utile dans notre cas, car certaines classes sont sous-représentées. Elle permet au modèle d'être entraîné sur un maximum d'échantillons tout en garantissant une évaluation plus fiable de ses performances. Chaque observation est utilisée à la fois pour l'entraînement et la validation, ce qui réduit les risques de biais liés à un simple découpage train/test.

3.5.1 Entraînement sur l'ensemble des données avec XGboost et LGBM

Après optimisation, les meilleurs paramètres obtenus pour XGBoost sont :

- `subsample = 1.0` : utilisation complète des données à chaque itération.
- `scale_pos_weight = 5` : compensation modérée du déséquilibre des classes.
- `reg_lambda = 5`, `reg_alpha = 0` : régularisation L2 modérée, aucune L1.
- `n_estimators = 500`, `max_depth = 4`, `min_child_weight = 3` : contrôle de la complexité des arbres pour limiter l'overfitting.
- `learning_rate = 0.01` : faible taux d'apprentissage nécessitant plus d'arbres, mais offrant plus de stabilité.
- `colsample_bytree = 0.7` : sélection aléatoire de 70% des features par arbre.

Pour LGBM :

- `subsample = 0.9`, `scale_pos_weight = 10` : sous-échantillonnage et compensation importante du déséquilibre.
- `reg_lambda = 1`, `reg_alpha = 0` : régularisation L2 légère.
- `n_estimators = 200`, `max_depth = 10`, `min_child_weight = 3`, `num_leaves = 31`.
- `learning_rate = 0.1`, `colsample_bytree = 0.7`.

TABLE 1 – Performances des modèles XGBoost et LGBM

Modèle	F1-score	Précision	Rappel
XGBoost	0,626	0,80	0,54
LGBM	0,625	0,66	0,627

Analyse : Ces résultats révèlent une bonne précision mais un rappel limité pour XGBoost, traduisant une certaine frilosité à prédire les classes rares. LGBM, avec une meilleure balance précision/rappel, semble plus sensible aux classes minoritaires, probablement dû à son `scale_pos_weight` plus élevé.

3.5.2 Undersampling et entraînement avec XGboost et LGBM

Face à une surreprésentation de la classe 1 en particulier, même avec un nombre élevé d'échantillons dans d'autres classes, nous avons appliqué une stratégie d'*undersampling progressif*. Nous avons :

- Conservé uniquement les échantillons des classes minoritaires (2, 3, 8, 10, 14, 16, 20, 21),
- Supprimé toutes les instances des classes majoritaires (1, 4, 5, 6, 7, 11, 12, 18),
- Réintégré les classes restantes (0, 9, 13, 15, 17, 19).

Résultats après undersampling concernant l'algorithme **XGBoost**, les hyperparamètres sont demeurés globalement stables. Cela suggère que le modèle conserve une bonne capacité d'adaptation, même sur un jeu de données réduit, sans nécessiter de modifications majeures de la stratégie d'optimisation.

En revanche, le modèle **LGBM** a nécessité quelques ajustements notables pour mieux s'adapter à la nouvelle distribution des données. Plus précisément :

- Le paramètre `num_leaves` a été augmenté à 50 afin de permettre au modèle de capturer des relations plus complexes.
- Le `learning_rate` a été réduit à 0,05, dans le but de favoriser un apprentissage plus progressif et précis.
- Le `colsample_bytree` a été abaissé à 0,5 pour limiter le surapprentissage en présence d'un volume de données plus restreint.

TABLE 2 – Performances des modèles XGBoost et LGBM après undersampling

Modèle	F1-score	Précision	Rappel
LGBM	0,5551	0,6973	0,5021
XGBoost	0,5332	0,6920	0,4914

Interprétation des ajustements Le *undersampling* a modifié la distribution des classes, réduisant la dominance de certaines classes fréquentes. Cela a eu plusieurs effets :

- **Baisse du F1-score de XGBoost** : plus grande difficulté à généraliser avec un nombre réduit d'exemples pour certaines classes.
- **Légère hausse du F1-score de LGBM** : meilleure capacité à exploiter les nouveaux équilibres de classe, grâce à une complexité d'arbre adaptée et une régularisation plus stricte.
- **Changement des hyperparamètres LGBM** : adaptation à la structure simplifiée du dataset avec moins de features pertinentes disponibles, d'où une réduction de `colsample_bytree` et un ralentissement du `learning_rate`.

3.5.3 Entraînement à l'aide de la méthode *MAPIE* [13] [14]

Estimation d'incertitude avec MAPIE La bibliothèque **MAPIE** (Model Agnostic Prediction Interval Estimation) constitue un outil performant pour l'estimation d'intervalles de prédiction dans les modèles de machine learning. Elle repose sur le cadre théorique de la *conformal prediction*, une approche statistique rigoureuse permettant de quantifier l'incertitude des prédictions, quel que soit l'algorithme utilisé. Cette méthode est dite *agnostique au modèle* : elle ne dépend ni de l'architecture interne du modèle, ni de la distribution des données, ce qui en fait un cadre particulièrement robuste et versatile.

D'un point de vue fréquentiste, MAPIE fournit des garanties de validité sur le long terme : pour un niveau de confiance fixé ($1 - \alpha$), elle construit des régions de prédiction — un intervalle dans le cas de la régression, ou un ensemble de classes dans le cas de la classification — de sorte que la probabilité que la véritable valeur cible appartienne à cette région soit supérieure ou égale à ce niveau de confiance :

$$P(y_{\text{inf}} \leq y \leq y_{\text{sup}}) \geq 1 - \alpha.$$

Intuition fondamentale. L'idée sous-jacente à la prédiction conforme est que plus un échantillon est atypique ou dissemblable par rapport aux données d'entraînement, plus l'incertitude de sa prédiction doit être grande. Cette notion est capturée par les *scores de non-conformité*, qui mesurent à quel point un point test diffère de ce que le modèle a appris. Ces scores peuvent prendre différentes formes selon la tâche : dans le cas de la classification, il peut s'agir par exemple de $1 - \text{probabilité prédite}$, tandis qu'en régression, on peut considérer l'écart absolu entre la prédiction et la valeur observée, ou encore des distances aux plus proches voisins.

Théorème fondamental. L'un des résultats clés de la prédiction conforme est qu'elle garantit, quel que soit le modèle de base utilisé et sous des hypothèses raisonnables sur la fonction de non-conformité, que le taux d'erreur — c'est-à-dire la proportion de cas où la vraie valeur ne se situe pas dans l'intervalle ou l'ensemble prédit — est asymptotiquement inférieur ou égal au niveau α choisi. Cette propriété reste valable même lorsque les données ne suivent pas une distribution connue, ce qui confère à MAPIE une applicabilité large, en particulier dans des contextes réels où les hypothèses de distribution sont difficilement vérifiables. (figure 5)

Appliquée ici au modèle *Random Forest*, cette approche a permis d'obtenir les meilleurs résultats observés jusqu'à présent. Le modèle Random Forest est un ensemble d'arbres de décision entraînés sur des sous-échantillons aléatoires des données. Il réduit

Algorithm 1: Inductive Conformal Prediction

input : Training data $(x_i, y_i)_{i \in I_{\text{train}}}$, calibration data $(x_i, y_i)_{i \in I_{\text{calib}}}$, $\mathcal{Y}$
the set of possible labels, significance level α , classifier $\mathcal{C}$,
non-conformity score r , new point x_{new} .
output: $C_{\text{conf}}(x_{\text{new}})$, prediction region at x_{new} .
Fit the classifier on training data $\mathcal{C}$ on $(x_i, y_i)_{i \in I_{\text{train}}}$.
Compute non-conformity score on calibration data $(r(x_i, y_i))_{i \in I_{\text{calib}}}$.
for $y \in \mathcal{Y}$ **do**
 let $\hat{y}_{\text{new}}$ the probability predicted by $\mathcal{C}$ for x_{new} to belong to class y ;
 compute $r(x_{\text{new}}, y) = 1 - \hat{y}_{\text{new}}$ and the associated p -value $p_y(x_{\text{new}})$.
end
return $C_{\text{conf}}(x_{\text{new}}) = \{y \in \mathcal{Y} \mid p_{y(x_{\text{new}})} \geq \alpha\}$

FIGURE 5 – Algorithme de Mapie

le surapprentissage en moyenne les prédictions des différents arbres, améliorant ainsi la robustesse et la précision. Les performances obtenues avec cette méthode sont prometteuses :

TABLE 3 – Performances Random Forest avec Mapie

Modèle	F1-score	Précision	Rappel
Random Forest + MAPIE	0,835	0,822	0,851

3.6 Entraînement avec un modèle RNN

Pour cet entraînement, nous réutilisons les embeddings générés précédemment à partir du fichier `TrainingText.txt`. Nous avons ensuite conçu une architecture neuronale reposant sur des couches denses, intégrant des mécanismes de régularisation pour améliorer la généralisation du modèle.

L'architecture mise en place est la suivante :

- Une couche dense de 512 neurones avec une activation ReLU et une entrée de dimension `input_dim`.
- Une couche de normalisation par lots (*Batch Normalization*).
- Une couche de régularisation *Dropout* avec un taux de 0,4.
- Une seconde couche dense de 256 neurones, suivie d'une normalisation par lots et d'un *Dropout* de 0,3.
- Une troisième couche dense de 128 neurones avec les mêmes traitements.
- Enfin, une couche de sortie dense avec autant de neurones que de classes, utilisant une activation sigmoïde.

L'utilisation de la fonction d'activation sigmoïde en sortie est essentielle dans le cadre d'une classification multi-étiquettes. Contrairement à une softmax qui force la somme des probabilités à 1 (cas multi-label), la sigmoïde permet d'évaluer de manière indépendante la probabilité d'appartenance de chaque classe, ce qui correspond mieux à notre problématique où un échantillon peut appartenir à plusieurs catégories simultanément.

Cette architecture a permis d'obtenir les meilleurs résultats jusqu'à présent :

TABLE 4 – Performances avec RNN

Modèle	F1-score	Précision	Rappel
RNN	0,771	0,899	0,713

3.7 Affinement du modèle de langage BERT [9] [10]

BERT (Bidirectional Encoder Representations from Transformers) est un modèle de langage pré-entraîné basé sur l’architecture des *Transformers*. Il apprend une représentation contextuelle des mots en considérant simultanément leur contexte gauche et droit, ce qui le rend particulièrement performant pour les tâches de classification de texte.

Dans notre cas, nous avons réutilisé l’ensemble des données du fichier `TrainingData.txt` et nous l’avons prétraité, ensuite nous avons divisées à 70% pour l’entraînement et 30% pour la validation. Le jeu de test est issu du fichier `TestData.txt`.

Lors des premiers essais, nous avons constaté un début de surapprentissage : pour 8 époques d’entraînement, la fonction de perte en validation augmentait à partir de la 4^e époque, passant de 0,130 à 0,136 à la 5^e, puis à 0,141 à la 6^e. Pour remédier à ce phénomène, deux stratégies ont été mises en place :

- **Early stopping** : arrêt anticipé de l’entraînement à la 3^e époque, afin de conserver le modèle avant que la suradaptation ne commence à se produire.
- **Régularisation via *weight decay*** : cette technique consiste à pénaliser les poids trop élevés dans le modèle en ajoutant un terme de régularisation $\lambda \|\theta\|^2$ à la fonction de perte. Cela encourage des poids plus petits et aide à généraliser les performances. Un *learning rate* de 2×10^{-5} a été utilisé. Github

Les performances obtenues sur les données de `TestData.txt` sont les suivantes :

TABLE 5 – Comparaison des performances selon les techniques de régularisation

Méthode	F1-score	Précision (test)	Rappel (test)
Early stopping	0,666 (test)	0,622	0,822
Weight decay	0,65 (test) / 0,71 (validation)	0,65	0,65

3.8 Conclusion de notre contribution

Analyse comparative des performances Le tableau ci-dessous présente un récapitulatif des performances des modèles explorés dans le cadre de cette étude, évalués à l’aide des métriques F1-score, précision et rappel. Globalement, le modèle **Random Forest combiné à MAPIE** se distingue nettement avec les meilleures performances sur les trois critères, atteignant un F1-score de **0,835**, une précision de **0,822** et un rappel de **0,851**. Ces résultats traduisent une excellente capacité de ce modèle à équilibrer justesse des prédictions et couverture des cas pertinents, tout en fournissant une estimation de l’incertitude, ce qui est particulièrement utile dans les contextes critiques.

Le modèle **RNN avec embeddings**, bien qu’affichant la précision la plus élevée (**0,899**), présente un F1-score global de **0,771** et un rappel de **0,713**, indiquant une forte tendance à prédire correctement lorsqu’il est sûr, mais une couverture plus limitée des cas réels.

Le modèle **BERT fine-tuné** obtient un bon rappel (**0,822**), mais sa précision (**0,622**) reste faible, ce qui réduit son F1-score à **0,666**. Cela suggère que, bien qu’il détecte efficacement les cas positifs, il produit également un nombre élevé de faux positifs.

Enfin, les modèles de type **gradient boosting** (*XGBoost* et *LGBM*), utilisés seuls ou avec undersampling, affichent des performances plus modestes. Leurs F1-scores varient de **0,626** pour XGBoost (baseline) à **0,533** pour XGBoost avec undersampling, mettant en évidence leur sensibilité au déséquilibre des classes et leur moindre gain via les techniques de undersampling.

En conclusion, les résultats obtenus confirment la supériorité de l’approche Random Forest enrichie par MAPIE dans notre cas d’usage, tant en termes de performance globale que de robustesse et d’interprétabilité. Néanmoins, les modèles neuronaux comme les RNN restent des concurrents solides, notamment lorsqu’une haute précision est recherchée.

TABLE 6 – Résumé des performances des modèles testés

Modèle / Méthode	F1-score	Précision	Rappel
Random Forest + Mapie	0,835	0,822	0,851
RNN avec embeddings	0,771	0,899	0,713
BERT FineTuned	0,666	0,622	0,822
XGBoost (baseline)	0,626	0,800	0,540
LGBM (baseline)	0,625	0,660	0,627
LGBM + undersampling	0,555	0,697	0,502
XGBoost + undersampling	0,533	0,692	0,491

4 Explicabilité avec SHAP [5] [6] [7] [11]

4.1 Motivation

Dans le cadre de la classification d'incidents aériens, il est essentiel de comprendre les raisons qui poussent le modèle à associer une description d'incident à une catégorie spécifique. Cela permet d'instaurer la confiance auprès des utilisateurs et de vérifier le bon comportement du système. Nous avons donc utilisé la méthode **SHAP** (SHapley Additive exPlanations), qui fournit une interprétation mathématiquement rigoureuse des décisions du modèle. Nous nous sommes orientés vers SHAP car c'est la méthode la plus adaptée et la plus répandue pour l'explicabilité des données textuelles.

4.2 Fondements théoriques

SHAP repose sur la théorie des jeux coopératifs et s'inspire directement des *valeurs de Shapley*, utilisées pour déterminer la contribution équitable de chaque joueur dans un jeu. Dans le contexte de l'apprentissage automatique, chaque **feature** est considérée comme un joueur, et la prédiction du modèle est la valeur du jeu.

La valeur de SHAP associée à une feature i est définie par la formule suivante :

$$\phi_i(f, x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f_{S \cup \{i\}}(x) - f_S(x)] \quad (1)$$

où :

- F est l'ensemble des caractéristiques,
- S est un sous-ensemble de F qui ne contient pas i ,
- $f_S(x)$ est la prédiction du modèle lorsqu'il ne prend en compte que les caractéristiques de S ,
- ϕ_i est la contribution de la feature i à la prédiction finale.

Cette formule satisfait plusieurs propriétés fondamentales :

- **Additivité** : la somme des valeurs SHAP est égale à la différence entre la prédiction et la valeur de base,
- **Consistance** : si l'importance d'une feature augmente, sa valeur SHAP ne diminue pas,
- **Symétrie** : deux features identiques ont la même valeur SHAP.

4.3 Visualisations SHAP

SHAP fournit différents types de visualisations pour mieux comprendre l'impact des variables :

- **Summary plot** : importance globale de chaque feature sur l'ensemble des prédictions,
- **Force plot** : explication individuelle d'une prédiction,
- **Text plot** : surlignage des mots clés directement dans le texte à expliquer.

4.4 Exemple d'explication SHAP sur une prédiction (XGBoost)

Nous présentons ci-dessous une explication SHAP détaillée de la prédiction du modèle XGBoost appliqué à l'incident numéro 40. Cette visualisation a été générée grâce à la fonction suivante :


```
shap.plots.text(shap_values_instance, display=False)
```

La figure 6 illustre les mots du texte d'entrée colorés selon leur contribution à la prédiction finale du modèle. L'interprétation des couleurs est la suivante :

- **Rouge** : mots ayant augmenté la probabilité d'appartenir à la classe prédite (valeurs SHAP positives),
- **Bleu** : mots ayant réduit cette probabilité (valeurs SHAP négatives),
- **Plus la couleur est saturée**, plus l'impact du mot sur la décision est significatif.

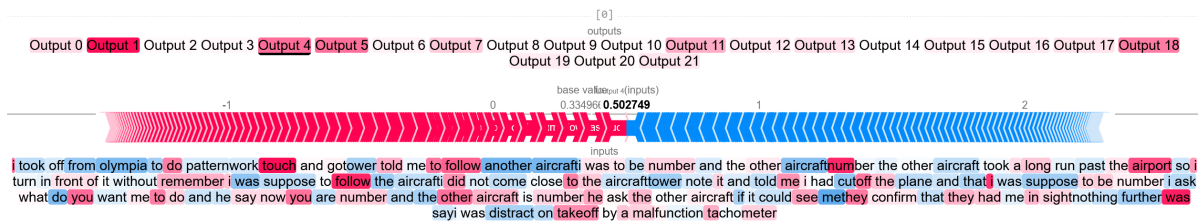


FIGURE 6 – Visualisation SHAP du texte numéro 40 pour la classe 4 — modèle XGBoost

L'analyse révèle que les mots suivants ont eu un poids décisif :

- En **rouge** : **touch**, **airport**, **follow**, **remember**, **takeoff**. Ces termes sont liés à des actions aériennes spécifiques (procédures de décollage, manœuvres, gestion de trajectoire) et ont orienté la prédiction vers une classe pouvant par exemple être relative à une violation de procédure ou de trajectoire (en réalité, nous n'avons pas la description des classes).
- En **bleu** : **aircraft**, **olympia**, **distract**, **took**, **another**, **plane**, qui ont une contribution plus diffuse ou atténuante, rendant la prédiction moins certaine.

Cette visualisation permet d'identifier avec clarté les **éléments textuels les plus influents** dans la prise de décision algorithmique, ce qui est crucial dans des contextes critiques tels que la sécurité aérienne.

En résumé, cette approche interprétable nous permet de :

1. Vérifier la cohérence des prédictions avec des éléments linguistiques pertinents,
2. Fournir une transparence utile pour les experts du domaine,
3. Détecter d'éventuels biais d'apprentissage ou faiblesses du modèle.

4.5 Conclusion de l'explicabilité

L'outil **SHAP** s'est révélé extrêmement utile pour expliquer chaque prédiction de manière transparente, en identifiant précisément les contributions positives et négatives de chaque mot dans un texte. Grâce à la fonction `shap.plots.text()`, nous avons pu visualiser de façon localisée les éléments influençant directement les décisions de nos modèles, ce qui s'avère fondamental pour renforcer la confiance dans le système — en particulier dans des domaines aussi critiques que la sécurité aérienne.

Cependant, notre expérience a aussi mis en lumière certaines **limites** de l'explicabilité. Dans plusieurs cas, la qualité des descriptions d'incidents en entrée s'est avérée médiocre : phrases très courtes, style télégraphique, ou encore formulations ambiguës. Ce caractère parfois *cryptique* du texte rend l'interprétation SHAP confuse — les mots surlignés peuvent sembler hors contexte, ou mal refléter l'intention du rédacteur.

Nous avons également mené des recherches pour déterminer s’il fallait ou non conserver certains types de mots dans l’analyse, notamment :

- les **mots de liaison** : *and, or, but*,
- les **pronoms personnels** : *I, you, me*,
- les **mots fonctionnels** très fréquents et peu informatifs en apparence : *a, to, do, was, had*, etc.

Malgré nos recherches, aucun consensus clair n’a été trouvé dans la littérature ou sur les forums spécialisés. Pourtant, ces mots pourraient jouer un rôle non négligeable dans la prédiction, soit en tant que signaux faibles, soit par leur association avec des contextes spécifiques — par exemple un style narratif propre à un type d’événement.

Ces observations renforcent l’idée qu’une explicabilité fiable repose d’abord sur des données **claires, cohérentes et bien prétraitées**. Une réflexion approfondie sur les étapes de nettoyage, de vectorisation, et de représentation textuelle est donc essentielle pour garantir la pertinence des interprétations fournies par des outils comme SHAP.

5 Prototype de classification des rapports d'incident

5.1 Présentation du site web

Le site web constitue l'interface principale permettant d'interagir avec le modèle de classification. Il permet à l'utilisateur de sélectionner un rapport d'incident à partir d'un ensemble de test comprenant **7077 textes** issus du fichier `Test.txt`. Chaque ligne correspond à un rapport d'incident anonymisé.

Capture d'écran de l'interface principale :

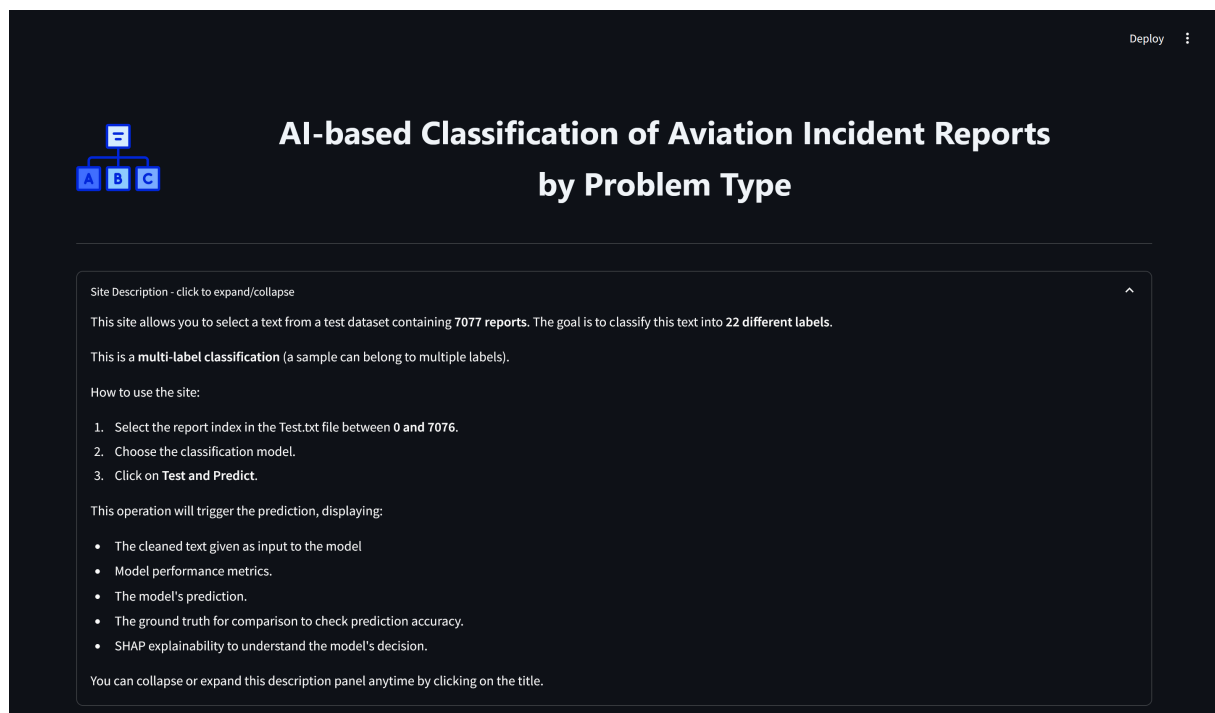


FIGURE 7 – Capture d'écran de l'interface principale du site web.

5.2 Fonctionnalités principales

L'utilisateur suit les étapes suivantes pour tester le modèle :

1. **Sélection du rapport** : l'utilisateur choisit un index compris entre 0 et 7076 pour extraire un rapport spécifique du jeu de test. Le rapport original s'affiche alors automatiquement dans l'interface.
2. **Choix du modèle de classification** : plusieurs modèles d'apprentissage supervisé ou profond sont proposés, chacun ayant des caractéristiques spécifiques en termes de performance et d'interprétabilité. On retrouve notamment les modèles : XGBoost, LGBM, un RNN (Réseau de Neurones Récurrent), un modèle BERT affiné via Fine-Tuning, ainsi que l'approche MAPIE (pour la quantification d'incertitude) combinée à une Random Forest.
3. **Lancement de la prédiction** : en cliquant sur le bouton `Test and Predict`, le système exécute l'ensemble de la chaîne de traitement. Le texte est d'abord

prétraité (nettoyé) puis transmis en entrée au modèle sélectionné. Une fois la prédiction effectuée, plusieurs éléments s'affichent dans l'interface :

- Le texte nettoyé utilisé comme entrée du modèle (*Cleaned Text*).
- Les métriques de performance du modèle (précision, rappel, F1-score, etc.).
- La ou les classes prédites par le modèle.
- La vérité terrain (ou *ground truth*) permettant une comparaison directe.
- Une visualisation par SHAP expliquant les décisions du modèle à travers les mots les plus influents dans la classification.

Capture d'écran de la sélection de l'index et du modèle :

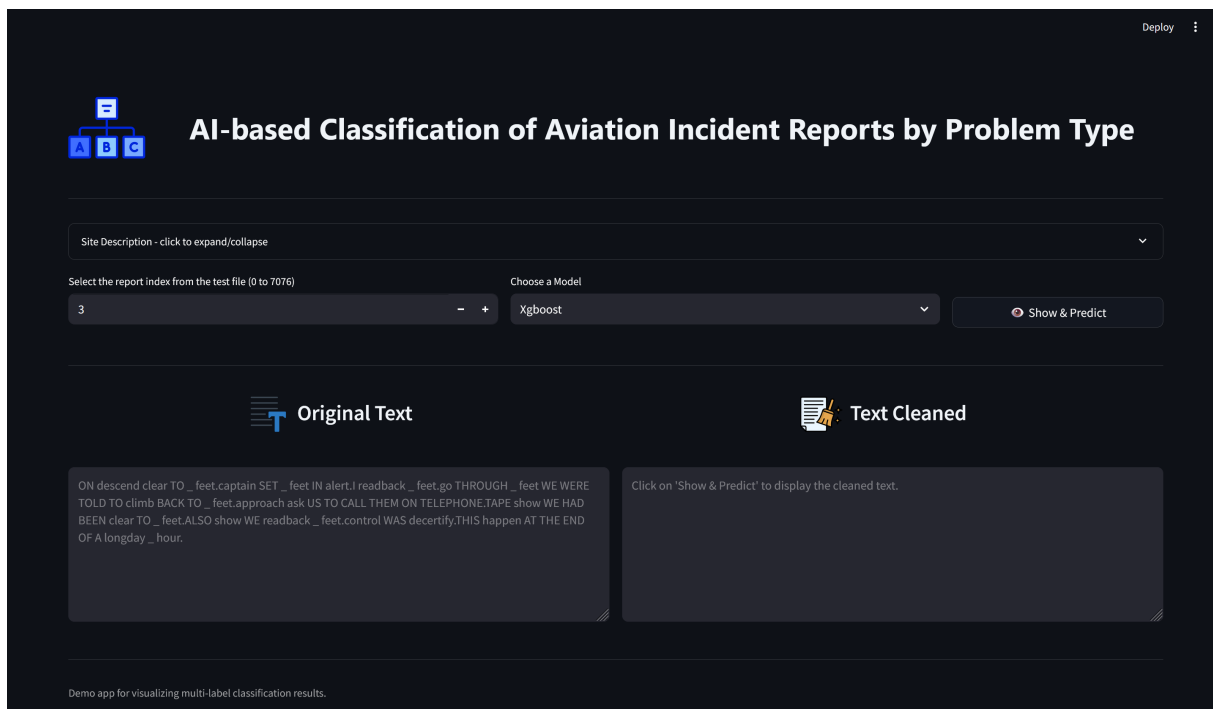


FIGURE 8 – Capture d'écran de la sélection de l'index et du modèle. Seul le texte original s'affiche automatiquement.

5.3 Résultats affichés après sélection du bouton "*Test and Predict*"

Une fois la prédiction lancée, plusieurs éléments sont présentés à l'utilisateur :

- Le texte nettoyé tel qu'il est présenté au modèle.
- Les métriques de performance du modèle (précision, rappel, F1-score, etc.).
- La prédiction du modèle (sous forme de liste de labels activés).
- Le **ground truth** (vérité terrain), permettant de comparer la prédiction à la réalité.
- Une visualisation SHAP pour expliquer la prédiction du modèle et identifier les mots les plus influents dans la décision.

Captures d'écran des résultats obtenus :

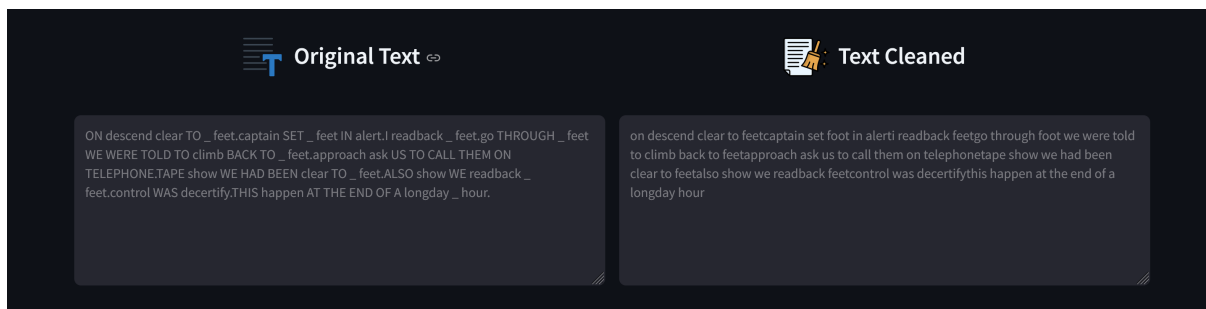


FIGURE 9 – Texte nettoyé utilisé en entrée du modèle.

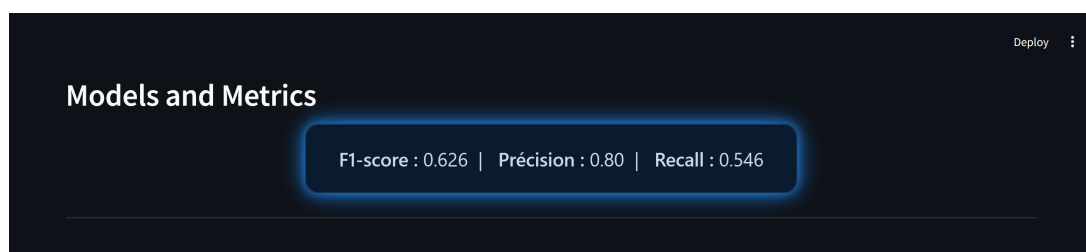


FIGURE 10 – Métriques de performance du modèle sélectionné.

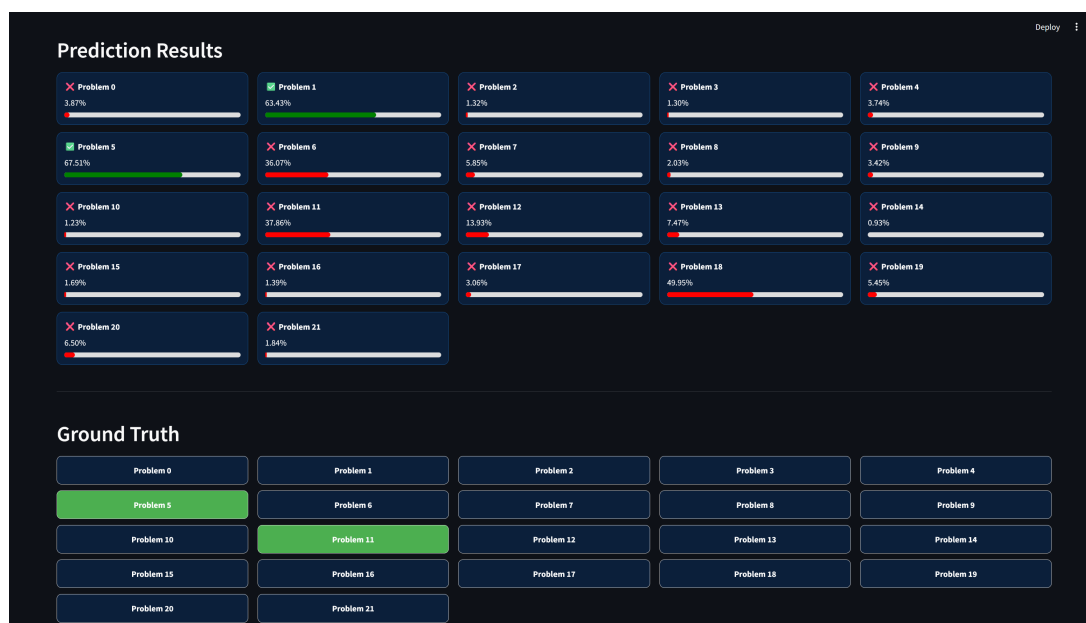


FIGURE 11 – Comparaison entre la prédiction du modèle et la vérité terrain (ground truth).

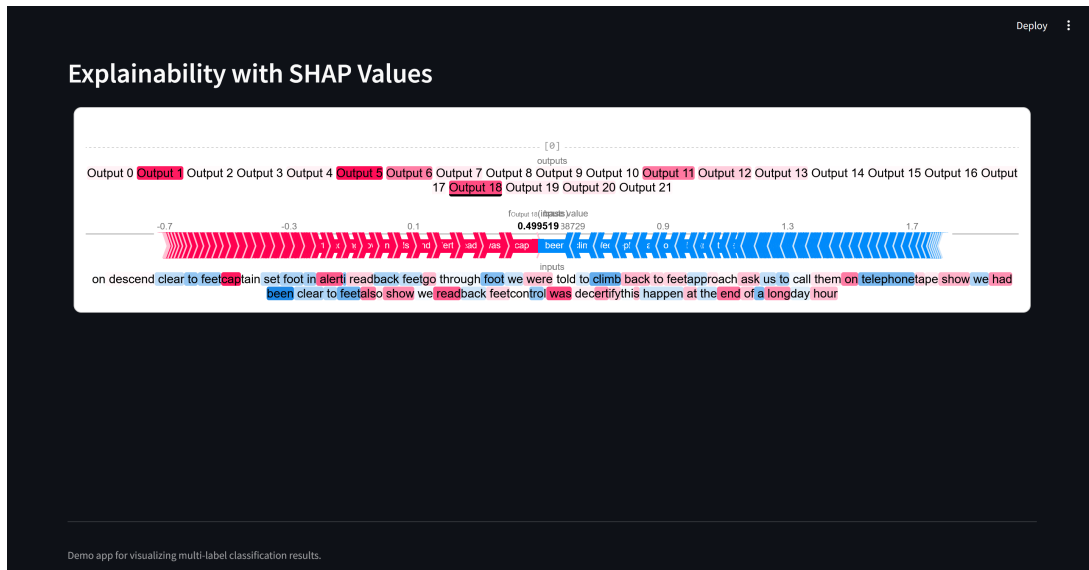


FIGURE 12 – Interprétation de la décision du modèle à l’aide de la méthode SHAP.

5.4 Résumé

Ce prototype constitue un outil de visualisation et de validation des résultats du modèle. Il met l’accent à la fois sur la performance et sur l’explicabilité, facilitant ainsi l’analyse critique par les utilisateurs non spécialistes du domaine de l’intelligence artificielle.

6 Conclusion et Perspectives

Ce projet a permis une exploration complète des méthodes de traitement automatique du langage naturel (NLP) appliquées à la classification multi-label de rapports d'incidents. Nous avons abordé de manière rigoureuse l'ensemble du processus, depuis le prétraitement des données textuelles, incluant nettoyage et vectorisation par embeddings avancés, jusqu'à l'implémentation et l'évaluation de modèles d'apprentissage supervisé, allant des approches classiques aux architectures neuronales profondes.

Les premières expérimentations avec des modèles traditionnels tels que *XGBoost* et *LightGBM* ont révélé des performances limitées, avec un F1-score avoisinant les 62–63 %, parfois dégradé par l'undersampling utilisé pour rééquilibrer les classes. Ces résultats suggèrent que des stratégies alternatives, comme l'oversampling ou des méthodes génératives de rééchantillonnage, pourraient offrir un meilleur compromis entre équilibre des classes et préservation de l'information.

Une avancée notable a été obtenue avec l'utilisation de **Random Forest** couplée à **MAPIE**, permettant d'intégrer une estimation d'incertitude tout en atteignant le meilleur F1-score de **0,835**. Cette combinaison s'est montrée particulièrement efficace dans des contextes à forte exigence de fiabilité, offrant à la fois robustesse, précision et interprétabilité.

Par ailleurs, les approches neuronales ont confirmé leur potentiel : le modèle **RNN avec embeddings** a obtenu un F1-score compétitif de **0,771**, avec une précision élevée (**0,899**) mais un rappel plus modeste. Ce contraste met en évidence la nécessité de choisir le modèle en fonction des priorités applicatives (précision vs rappel). Le modèle **BERT fine-tuné**, quant à lui, a atteint un bon rappel (**0,822**) mais souffre d'un déficit de précision (**0,622**), limitant son efficacité globale (F1-score de **0,666**).

Il est également important de souligner que la qualité des données d'origine n'était pas idéale : les rapports d'incident présentaient une grande hétérogénéité en termes de longueur, une grammaire souvent imparfaite, des erreurs de segmentation lexicale, une narration ambiguë, ainsi qu'une anonymisation systématique des chiffres. Ces particularités ont constitué un défi majeur pour le traitement automatique et ont nécessité un prétraitement linguistique rigoureux afin d'atténuer leur impact négatif.

Plusieurs pistes d'amélioration sont envisageables : l'exploration d'architectures plus sophistiquées telles que LSTM ou GRU [12], l'intégration de traitements linguistiques avancés (correction orthographique, segmentation lexicale), ainsi que l'approfondissement des stratégies d'échantillonnage, notamment l'oversampling ou des méthodes génératives, pour mieux traiter le déséquilibre des classes [4].

En résumé, ce projet a enrichi notre compréhension des défis liés à la classification textuelle sur des données industrielles complexes, mettant en évidence les limites des méthodes classiques et la puissance des approches neuronales couplées à des techniques d'interprétabilité. Il confirme l'importance d'une démarche méthodologique rigoureuse pour concevoir des systèmes robustes, fiables et adaptés aux exigences à la fois scientifiques et industrielles.

Références

- [1] Kamal Taha , Paul D. Yoo, Chan Yeun, Dirar Homouz, Aya Taha, A comprehensive survey of text classification techniques and their research applications : Observational and experimental insights, Computer Science Review Volume 54, November 2024, 100664.
- [2] NI Xiaomei , WANG Huawei , CHEN Lingzi , and LIN Ruiguan, Classification of aviation incident causes using LGBM with improved cross-validation, IEEE Journal of Systems Engineering and Electronics Vol. 35, No. 2, April 2024, pp.396 – 405.
- [3] Xiao Jing , Akul Chennakesavan, Chetan Chandra, Mayank V. Bendarkar, Michelle Kirby, Dimitri N. Mavris, BERT for Aviation Text Classification, June 2023, DOI :10.2514/6.2023-3438
- [4] Maren David Dangut, , Zakwan Skaf, Ian K. Jennions, Handling imbalanced data for aircraft predictive maintenance using the BACHE algorithm, Computer Science Review Volume 123, July 2022, 108924
- [5] Francesco Dallanocce, Explainable AI : A Comprehensive Review of the Main Methods, <https://medium.com/@dallanocce.fd/explainable-ai-a-complete-summary-of-the-main-methods-a28f9ab132f7>, Accessed : 2025-05-16
- [6] <https://data.world/resources/compare/explainable-ai-tools/> Accessed : 2025-05-15
- [7] Melkamu Abay Mersha, Mesay Gameda Yigezu, Jugal Kalita, Evaluating the Effectiveness of XAI Techniques for Encoder-Based Language Models, arXiv :2501.15374, Submitted on 26 Jan 2025
- [8] Hugging Face, Fine-tuning with custom datasets, https://huggingface.co/transformers/v3.2.0/custom_datasets.html, Accessed : 2025-03-24
- [9] Abdurhman Fayad, Github, <https://gist.github.com/AFayad0/ca196fc2642cbb01a10b2f3fb2d1e9a4>, Accessed : 2025-03-26
- [10] Abdurhman Fayad, Medium, FineTuningBERTforaMulti-LabelClassificationProblemonColab Nov 5, 2023
- [11] Sameer Danave, Explainable AI : Seven Tools and Techniques for Model Interpretability, <https://dzone.com/articles/explainable-ai-7-tools-and-techniques-for-model>, Jul. 01, 24
- [12] Aziida Nanyonga, Graham Wild, Impact of Dataset Size Data Source on Aviation Safety Incident Prediction Models with Natural Language Processing, 2023 Global Conference on Information Technologies and Communications (GCITC) Karnataka, India. Dec 1-3, 2023
- [13] Candice Moyet, Estimer l'incertitude de manière adaptative en régression avec MAPIE, <https://www.quantmetry.com/blog/incertitude-adaptative/> , 06/11/2023
- [14] Vianney Taquet, Vincent Blot, Thomas Morzadec, Louis Lacombe, Nicolas Brunel, MAPIE : an open-source library for distribution-free uncertainty quantification, arXiv :2207.12274v1 [stat.ML] 25 Jul 2022